

FACULTY OF SCIENCE, ENGINEERING AND COMPUTING

School of Computer Science & Mathematics

BSc DEGREE IN

Computing Science in Software Engineering (TOP UP)

PROJECT INTERIM REPORT

Name: I.D. Nuwan Kumara

ID Number: E157735

Chili Management System

Build

14th of August 2025

Supervisor: Ms. Virajini Godapitiya

Kingston University London

Did you discuss and agree the viability of your project idea with your supervisor?	Yes/No
Did you submit a draft of your proposal to your supervisor?	Yes/No
Did you receive feedback from your supervisor on any submitted draft?	Yes/No

Abstract

The chilli industry faces challenges due to reliance on manual processes, supply chain inefficiencies, lack of real-time pricing data, and limited transparency. This project addresses these issues by developing a Chilli Management System, a digital platform designed to optimize chilli production, supply, and pricing. The system integrates user authentication and role-based dashboards for administrators, suppliers, and buyers, enabling efficient management and access to relevant information.

A key component of the system is a price prediction system that utilizes machine learning algorithms to forecast chilli prices based on historical data and market trends. This feature aims to provide price visibility and support fair trade decisions. The system also includes inventory management functionality to track stock levels and a supplier-buyer communication module to facilitate direct interaction and negotiations.

The system is designed using a multi-tiered architecture, with a focus on user-friendly interfaces and robust data management. The implementation involves technologies such as [I'll need to specify technologies used, e.g., Python, a web framework, and a database]. The system's performance and usability will be evaluated through testing and user feedback.

The Chilli Management System is expected to enhance market efficiency, improve price stability and forecasting, increase transparency, and reduce transaction costs. By digitizing the chilli market and providing tools for better decision-making, the project contributes to the development of a more efficient, transparent, and profitable chilli supply chain.

Contents

1. Introduction & Literature Review	6
1.1 Introduction	6
1.2 Background and Motivation.....	6
1.3 Problem in brief.....	8
1.4 Aim & Objectives.....	8
1.4.1 Aim	8
1.4.2 Objectives	9
1.5 Scope	9
1.6 Deliverables.....	10
1.7 Literature Review	12
2. Analysis.....	17
3. Design	24
3.1 Design Techniques	24
3.2 System Overview	27
4. Product Implementation.....	31
5. Validation.....	31
6. Critical Review & Conclusion	36
6.1 Closing executive summary	42
6.2 Conclusion.....	42
References / Bibliography.....	43

List of Figures/Tables

Figure 1 – Flow Chart	19
Figure 2 – Activity Diagram	20
Figure 3 – Sequence Diagram	21
Figure 4 – Use Case Diagram	22
Figure 5 – System Architecture	23
Figure 6 – Data Structure	23
Figure 7 – System Dashboard	24
Figure 8 – Price Predictor	24
Figure 9 – Predictions	25
Figure 10 – Post a Sale	25
Figure 11 – Error State	40
Figure 12 – Working Example	41
Table 1 – SWOT Analysis	12
Table 2 – Use Cases	13

Glossary of Terms

GDP - Gross Domestic Product

ARIMA - Autoregressive Integrated Moving Average

ANNs - Artificial Neural Networks

ML - Machine learning

CSS - Cascading Style Sheets

UI - User Interface

LSTM - Long Short-Term Memory

CRUD – Create, Read, Update, Delete

DTO – Data Transfer Object

FFNN – Feed-Forward Neural Network

GRU – Gated Recurrent Unit

IoT – Internet of Things

JSON – JavaScript Object Notation

KPI – Key Performance Indicator

LM – Levenberg–Marquardt (training algorithm for ANN)

MVP – Minimum Viable Product

RLS – Row-Level Security

RMSE – Root Mean Square Error

RNN – Recurrent Neural Network

SCG – Scaled Conjugate Gradient (training algorithm for ANN)

SSL – Secure Sockets Layer

SWOT – Strengths, Weaknesses, Opportunities, Threats

UML – Unified Modeling Language

WFP – World Food Programme

1. Introduction & Literature Review

1.1 Introduction

The agricultural sector underpins many economies worldwide-and in Sri Lanka, it not only employs a large share of the workforce but also makes a significant contribution to GDP. Despite its importance, farmers and consumers alike face substantial hardship due to volatile crop prices. In the chilli market especially, manual inventory controls, opaque supply chains, and the lack of real-time pricing data compound these challenges, often resulting in economic instability and inefficient decision-making.

To help stabilize prices and streamline operations, this project proposes a modern, data- driven Chilli Management System. By leveraging advanced data-mining techniques and machine-learning algorithms-proven in numerous studies to outperform traditional time- series models like ARIMA-the system will deliver accurate price forecasts, automated inventory tracking, and seamless transaction management. Core functionalities will include role-based access for administrators, suppliers, and buyers; a neural-network- based price prediction engine; an intuitive inventory dashboard; and an integrated communication module to foster transparent, direct interactions.

1.2 Background and Motivation

The agricultural sector is a cornerstone of many economies, including that of Sri Lanka, where it employs a significant portion of the labor force and contributes substantially to the national GDP. However, this sector faces significant challenges, particularly substantial price fluctuations in agricultural products. These fluctuations can distress farmers and consumers and lead to economic instability.

Traditional methods in the agricultural market, especially in the chilli sector, rely heavily on manual processes. This can lead to complications in inventory tracking,

price prediction, and transaction management. Inefficient supply chain management, the absence of real-time pricing data, and limited transparency between suppliers and buyers further exacerbate these issues.

To address these challenges, there's a growing need for advanced technological solutions, specifically the application of data mining techniques and machine learning algorithms. These technologies offer the potential to analyze complex agricultural data, predict price variations, and optimize supply chain processes.

Various studies have demonstrated the effectiveness of machine learning in agricultural price prediction. Artificial Neural Networks (ANNs), for instance, have been successfully applied to forecast the prices of various crops, often outperforming traditional time-series models like ARIMA. These models can capture non-linear relationships and provide more accurate predictions, which are crucial for effective decision-making by farmers, suppliers, and other stakeholders.

This project is motivated by the need to modernize the chilli supply chain by developing a chilli management system. The system will integrate key functionalities, including:

- User authentication and role-based dashboards for different users (admin, supplier, and buyer).
- A price prediction system using machine learning algorithms.
- Inventory management to track stock levels.
- A communication module to facilitate direct interaction between suppliers and buyers.

By addressing the issues of price fluctuation, inefficient supply chains, and lack of transparency, the project aims to enhance market efficiency, improve price stability, reduce transaction costs, and support data-driven decision-making in the chilli industry.

1.3 Problem in brief

The core issue is the inefficiency and lack of optimization in the chilli supply chain, specifically concerning production, supply, and pricing. This inefficiency arises from several factors:

- **Reliance on manual processes:** Traditional methods in the chilli market depend heavily on manual operations. This makes it hard to keep track of inventory, predict price changes, and manage transactions effectively.
- **Supply chain inefficiencies:** The supply chain lacks smooth operation, leading to problems in getting chilli from producers to consumers.
- **Lack of real-time pricing data:** There's a shortage of up-to-date information on how much chilli is worth at any given time. This makes it difficult for sellers and buyers to agree on fair prices.
- **Limited transparency:** There's a lack of openness in the dealings between those who provide the chilli and those who buy it. This can create distrust and make it harder to build strong business relationships.

In short, the absence of a good management system means the chilli business isn't working as well as it could. This project aims to fix that by creating a system that makes things more organized, predictable, and fair for everyone involved.

1.4 Aim & Objectives

1.4.1 Aim

The project aims to digitally transform the chilli supply chain by creating efficient and transparent processes for suppliers, buyers, and administrators.

1.4.2 Objectives

- Develop a secure and user-friendly digital Platform for Admins, Suppliers, and Buyers.
- Apply machine learning algorithms to forecast chilli prices based on market trends and historical data.
- Develop price visibility to help buyers and suppliers make fair trade decisions.
- Facilitate direct communication between suppliers and buyers for better negotiations.
- Monitoring system performance and user activity for continuous improvements.

1.5 Scope

The Chilli Management System will:

- Include
 - User authentication & role-based dashboards (Admin, Supplier, Buyer)
 - Price prediction using machine learning (ANNs, historical data, market trends)
 - Real-time inventory management
 - Supplier–buyer communication module
 - Data storage, analytics, and reporting tools
 - Web-based interface accessible via desktop and mobile
- Exclude
 - Physical chilli transport/logistics management
 - Direct financial transaction processing (e.g., payment gateways) at initial stage
 - Government licensing or regulation enforcement modules

The system aims to digitally transform the chilli supply chain in Sri Lanka by improving transparency, efficiency, and price stability, while empowering smallholder farmers and traders.

SWOT Analysis is as follows;

Strengths

- Machine learning based price prediction for better decision-making.
- Real-time inventory tracking to avoid shortages or oversupply.
- Centralized platform for supplier–buyer communication.
- Supports transparency and trust.

Weakness

- Initial dependency on reliable internet access.
- Limited historical datasets for training accurate models.
- Need for technical training for non-digital stakeholders.

Opportunities

- Can expand to other crops beyond chilli.
- Potential government adoption as an official price monitoring tool.
- Data insights for agricultural policy.

Threats

- Cyber security threats (data breaches, hacking).
- Resistance to adoption from traditional traders.
- Market fluctuations due to climate change beyond prediction accuracy.
- Competition from other agri-tech platforms.

1.6 Deliverables

Upon completion of the **Chilli Management System** project, the following will be delivered:

i. Fully Functional Web Application

- Role-based access for Administrators, Suppliers, and Buyers.
- Machine learning–powered chilli price prediction module.
- Real-time inventory management dashboard.
- Supplier–buyer communication and transaction facilitation.
- Data analytics and reporting tools for market trends.

ii. **GitHub Repository**

- Complete source code for backend (FastAPI, Python) and frontend (React, Tailwind CSS).
- Configuration files, database schema, and trained machine learning model files.
- Version history for code review and future enhancements.

iii. **Technical Documentation**

- Software Requirements Specification (SRS).
- System design diagrams (architecture, ER diagrams, data flow).
- API specifications and integration details.
- Machine learning model development report.

iv. **Testing & Evaluation Reports**

- Functional and non-functional test case results.
- Bug tracking and resolution details.
- Model accuracy and performance evaluation metrics.

v. **Deployment Package**

- Cloud-hosted live system with SSL security and domain.
- Staging environment for testing and future updates.
- Deployment and maintenance guide.

vi. **User Support Materials**

- User manuals for Admin, Supplier, and Buyer.
- Quick-start guides and FAQs to support adoption.

Outcome:

A secure, scalable, and user-friendly digital platform that improves chilli market efficiency, enhances transparency, supports fair pricing, and empowers farmers and traders with real-time data.

1.7 Literature Review

Forecasting agricultural commodity prices and digitising market interactions are active research areas because price volatility and lack of market transparency harm producers and buyers alike. Machine learning (ML) and deep learning (DL) methods are increasingly applied to short- and medium-term price forecasting and outperform classical time-series models on many datasets, especially when nonlinear relationships exist and additional features (weather, input costs, seasonality) are available.

Price-forecasting methods for agricultural commodities

Traditional time-series methods

ARIMA and other Box-Jenkins models remain baseline methods for univariate price forecasting and perform well for short-term linear patterns. Their interpretability and low-data requirements make them useful as baselines. However, ARIMA struggles with nonlinearity and with exogenous variables unless extended (SARIMAX, ARIMAX).

Machine learning and deep learning approaches

A wide range of ML/DL methods have been applied to commodity pricing: feed-forward neural networks (FFNN), time-delay neural networks (TDNN), recurrent models (RNN, LSTM, GRU), ensemble tree methods (XGBoost, Random Forest), and hybrid models (ARIMA+LSTM, ARIMA+ANN). Systematic reviews show ML/DL and ensemble models often give superior accuracy versus ARIMA when sufficient historical data and feature sets are available. Recent large-scale comparisons (including 2010–2024 datasets) confirm deep models (LSTM, GRU) and ensembles frequently outperform classical models, particularly for volatile series.

Region- and crop-specific studies — green chilli in Sri Lanka

Crop- and country-specific studies are important because market dynamics (seasonality, local market structure, policy) differ. For example, Basnayake et al.

(2022) applied ANN variants (TDNN, FFNN with LM and SCG training) to weekly green-chilli prices in Sri Lanka and demonstrated the practical usefulness of ANNs for that market. Their findings support the viability of neural networks for chilli price forecasting in the Sri Lankan context.

Features, data sources, and preprocessing

Accuracy depends heavily on input features and quality of data. Typical inputs include historical wholesale/retail prices, volumes, seasonality indicators, local weather (rainfall, temperature), policy events, transportation/market fees, and macroeconomic indicators. Public and private data sources used in studies include national market price reports, WFP/UN or World Bank price datasets, and scraped marketplace posts; synthetic and curated Sri Lanka datasets are also available on platforms such as Kaggle. Preprocessing steps (missing-value handling, outlier removal, differencing, normalization) and feature engineering (lags, rolling statistics, calendar features) are critical.

Evaluation metrics and model validation

Common evaluation metrics for regression/forecasting tasks include MAE, RMSE, MAPE and MASE. Proper validation practices—time series cross-validation (walk-forward), retaining an out-of-time test set, and reporting uncertainty (prediction intervals)—are emphasized in high-quality studies. Overfitting is a persistent risk with neural nets, so model regularization, early stopping, and careful hyperparameter search (grid/Bayesian search) are recommended.

Inventory management and operational systems in agriculture

Real-time inventory control in agri applications is a growing practice, often implemented within ERP/AgriERP systems to reduce waste, manage seasonality, and optimize restocking (demand forecasting + alerts). Systems that combine demand forecasting with inventory rules (reorder point, safety stock) produce operational benefits by aligning stock with seasonal demand. Commercial agri-

ERP products highlight features like batch tracking and low-stock alerts which are directly relevant to your proposed inventory module.

Digital platforms, transparency, and supply-chain governance

Digital traceability, smartphone apps, IoT sensors, and blockchain have been studied as ways to increase transparency and empower smallholders. Reviews show digital platforms can improve price discovery and traceability but adoption is constrained by connectivity, costs, and governance arrangements; inclusive governance and attention to smallholder incentives are critical for equitable outcomes. Studies of agri-food digital governance emphasise that design must consider value distribution, data ownership, and farmer participation.

Tools, deployment, and practical stacks used in literature & industry

Papers and practice notes illustrate common technical stacks:

Model development: TensorFlow / Keras, scikit-learn, Pandas/NumPy.

API & serving: lightweight REST frameworks (FastAPI) with automatic OpenAPI docs for maintainability and integration

Databases: PostgreSQL for transactional data; cloud storage for model artifacts and backups.

Deployment: cloud VMs or managed services (AWS/GCP) with staging/production separation and SSL.

These tool choices match recent practice for academic prototypes that aim for reproducibility and a path to production

Gaps in the literature and open problems

- i. **Data scarcity & quality for niche crops/regions.** Some crops (e.g., green chilli) and local markets have limited granular historical datasets; many studies use aggregated or short time series which constrains model complexity.

- ii. **Explainability and trust in ML forecasts.** Producers and traders need transparent explanations to trust model outputs; black-box DL models raise adoption barriers.
- iii. **Inclusion of smallholders and digital divide.** Platforms that don't consider connectivity, literacy, and governance risk excluding small farmers.
- iv. **Operational integration.** Bridging forecasting models with operational inventory rules, messaging, and negotiation tools remains under-explored in academic prototypes (many studies focus on forecasting only).

Proposed Chilli Management System addresses these gaps

Region-specific model development: Train ANN/LSTM (with careful regularization and walk-forward validation) on Sri Lanka market data (government price reports, local weekly markets, WFP datasets, Kaggle sources) to mitigate the data-scarcity issue and demonstrate performance on green-chilli prices. Basnayake et al. (2022) provide a direct empirical precedent for the Sri Lankan chilli market.

Hybrid baseline & interpretability: Use ARIMA or simple linear baselines plus neural nets; provide feature-importance analyses (e.g., SHAP for tree models or surrogate explainers for neural nets) and prediction intervals to increase stakeholder trust.

Operational integration: Link forecasts to inventory rules (reorder alerts, restock suggestions) and the supplier-buyer communication module so forecasts have immediate operational value — an area less covered in literature.

Deployment & reproducibility: Use reproducible artifacts (GitHub repo, Docker/container instructions, FastAPI with OpenAPI docs) to ensure your work is auditable and extendable — good scientific practice backed by tool docs.

Inclusivity considerations: Design UX for low-bandwidth/mobile users, data-ownership policies, and simple explanatory UI elements so smallholders can adopt the system — addressing governance and adoption concerns flagged in reviews.

Key references

Basnayake, B.R.P.M., et al. (2022). *An Approach for Prediction of Weekly Prices of Green Chili in Sri Lanka: Application of Artificial Neural Network Techniques*. Journal of Agricultural Sciences – Sri Lanka.

Systematic reviews of ML for commodity pricing: MDPI *Predicting Prices of Staple Crops Using Machine Learning* (2023).

Recent comparative/deep learning studies: *Enhancing agricultural commodity price forecasting with deep learning* (Nature/Scientific Reports, 2025).

Food supply chain governance & digital traceability: PMC review (2025).

Data sources: WFP/World Food Programme and Kaggle Sri Lanka crop prices datasets.

Practical tool references: FastAPI OpenAPI docs.

2. Analysis

Usage of Appropriate Analysis Techniques

In developing the Chilli Management System, a combination of analysis techniques was applied to ensure a comprehensive understanding of the problem space, user needs, and system requirements.

Clear Problem Definition

The primary problem was identified as inefficiency and lack of transparency in the chilli supply chain due to:

- Reliance on manual processes.
- Absence of real-time pricing data.
- Poor supplier–buyer communication.
- Supply chain inefficiencies causing delays and unfair pricing.

This clear definition provided a focused scope for solution design.

SWOT Analysis

S – Strengths	Machine learning based price prediction for better decision-making. Real-time inventory tracking to avoid shortages or oversupply. Centralized platform for supplier–buyer communication. Supports transparency and trust
W – Weakness	Initial dependency on reliable internet access. Limited historical datasets for training accurate models. Need for technical training for non-digital stakeholders
O – Opportunities	Can expand to other crops beyond chilli. Potential government adoption as an official price

	monitoring tool. Data insights for agricultural policy.
T – Threat	Cyber security threats (data breaches, hacking). Resistance to adoption from traditional traders. Market fluctuations due to climate change beyond prediction accuracy. Competition from other agri-tech platforms

Table 1 – SWOT Analysis

Use Cases

Administrator: Manage Users	
Use Case ID	UC-A1
Actor	Administrator
Goal	To manage system users, including suppliers and buyers.
Preconditions	Administrator is logged into the system
Main Flow	Admin opens the “User Management” module. System displays list of registered users with status (active/inactive). Admin can add, edit, deactivate, or delete user accounts. System saves changes and updates user access rights.
Postconditions	User records are updated in the database
Administrator: Monitor Inventory	
Use Case ID	UC-A2
Actor	Administrator
Goal	To oversee system monitor chilli inventory levels

Preconditions	Admin is logged into the system. Transaction and inventory data is up-to-date.
Main Flow	Admin opens “System Reports” dashboard. System displays current inventory levels for each supplier. System displays all completed and ongoing transactions. Admin reviews for anomalies, fraud, or unusual patterns.
Postconditions	Admin ensures smooth operations and fair trade
Supplier: Manage Inventory	
Use Case ID	UC-S1
Actor	Supplier
Goal	To add, update, or remove chilli stock details in the system
Preconditions	Supplier is registered and logged into the system Supplier has chilli stock available
Main Flow	Supplier logs in and selects the “Inventory Management” module. System displays current stock records. Supplier can: Add new stock entries with quantity, quality grade, and price. Update existing stock details. Remove stock that is sold out or unavailable.

	System saves updates to the database.
Postconditions	Updated inventory data is visible to buyers and administrators in real time
Supplier: View Price Predictions	
Use Case ID	UC-S2
Actor	Supplier
Goal	To access predicted chilli prices to plan selling strategies.
Preconditions	Supplier is logged into the system. Historical and market data is available for ML prediction
Main Flow	Supplier navigates to the “Price Prediction” module. System retrieves historical and real-time market price data. ML model processes the data and forecasts future prices. Predicted prices are displayed with a confidence percentage.
Postconditions	Supplier can decide whether to sell now or store stock for a better price
Supplier: Communicate with Buyers	
Use Case ID	UC-S3
Actor	Supplier
Goal	To interact directly with buyers for negotiations or clarifications
Preconditions	Supplier is logged into the system.

	Buyer has expressed interest in a product
Main Flow	Supplier opens the “Messaging” module. System displays a list of buyer inquiries. Supplier selects a conversation and responds with pricing, availability, or delivery details. Supplier and buyer agree on terms
Postconditions	Conversation history is stored for reference and dispute resolution
Supplier: Process Orders	
Use Case ID	UC-S4
Actor	Supplier
Goal	To receive, confirm, and fulfill buyer orders.
Preconditions	Supplier has active stock listings. Buyer places an order
Main Flow	System notifies supplier of a new order. Supplier reviews order details. Supplier confirms availability and delivery schedule. Order status is updated to “Confirmed” in the system. After delivery, supplier marks order as “Completed.”
Postconditions	Transaction details are stored for reporting and payment processing
Buyer: View and Purchase Chilli Stock	

Use Case ID	UC-B1
Actor	Buyer
Goal	To view available chilli stock, compare prices, and make purchases
Preconditions	Buyer is registered and logged into the system. Supplier listings and prices are available in the database.
Main Flow	Buyer logs in to the system. Buyer selects “Marketplace” from the dashboard. System displays available chilli stock, prices, and supplier ratings. Buyer filters or sorts listings (e.g., by price, quantity, location). Buyer selects a supplier and reviews stock details. Buyer places an order through the system. Payment and delivery arrangements are confirmed.
Postconditions	Order is recorded in the system. Buyer and supplier receive order confirmation notifications.
Buyer: View Price Predictions	
Use Case ID	UC-B2
Actor	Buyer
Goal	To view predicted chilli prices for informed purchasing

	decisions
Preconditions	Buyer is logged into the system. Historical and market data is available for ML prediction.
Main Flow	Buyer navigates to “Price Prediction” module. System retrieves and processes historical chilli price data. Machine learning model forecasts future prices. Predicted price range and confidence score are displayed.
Postconditions	Buyer uses the forecast data to decide on purchase timing

Table 2 – Use Cases

Use cases helped clarify functional boundaries and user-system interactions.

User Stories

As a supplier, See predicted chilli prices so I can decide the best time to sell.

As a buyer, Compare current market prices across suppliers so I can make cost-effective purchases.

As an administrator, Monitor inventory and transaction logs so ensure fair trade and system reliability.

User stories ensured that development priorities aligned with real stakeholder needs.

3. Design

3.1 Design Techniques

To ensure the Chilli Management System is designed systematically and aligns with user requirements, several established design techniques were applied. These techniques supported both high-level architectural planning and detailed process design, ensuring the system is scalable, maintainable, and user-friendly.

A Waterfall with Iterative Refinement model was used. The initial phases (requirements gathering, system design) follow a waterfall approach to ensure a solid foundation. Iterative refinements are applied during development and testing to incorporate user feedback, particularly for the UI and machine learning models. This hybrid approach ensures clear milestones while allowing for agile responsiveness to stakeholder needs.

Flow charts were developed to visualize core functional flows,

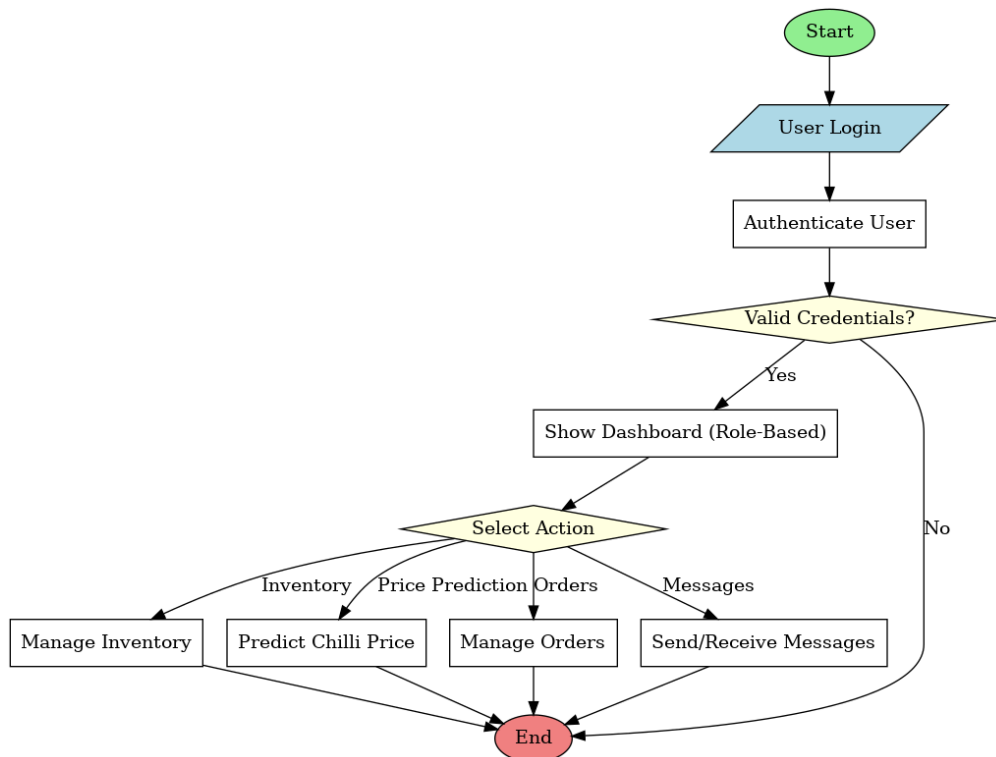


Figure 1 – Flow Chart

Flow charts ensured a clear understanding of process logic before coding began.

Unified Modeling Language (UML) activity diagrams were used to show the sequence of actions for critical tasks:

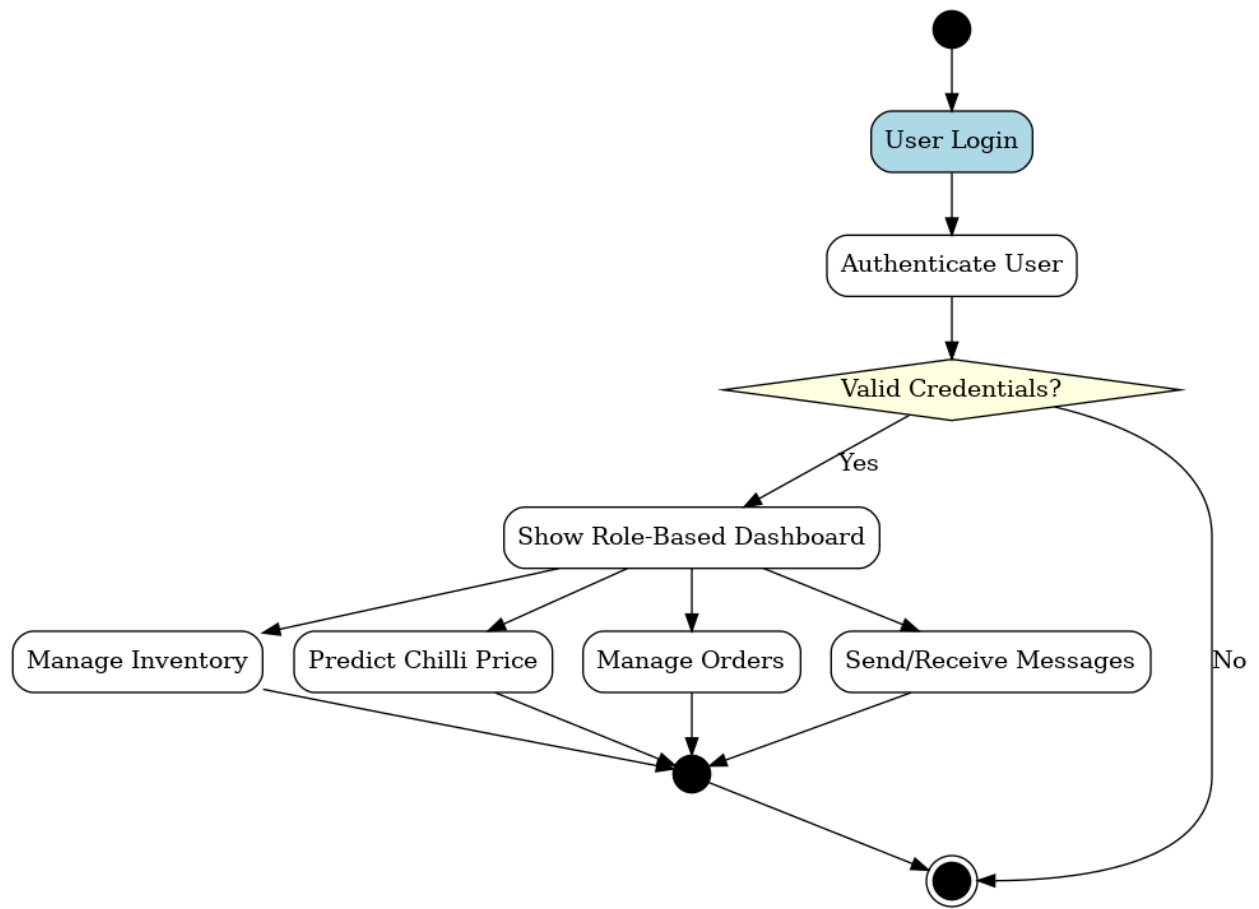


Figure 2 – Activity Diagram

Activity diagrams clarified parallel and conditional flows, making them invaluable for backend logic planning.

Sequence diagrams were created for time-dependent interactions between system components.

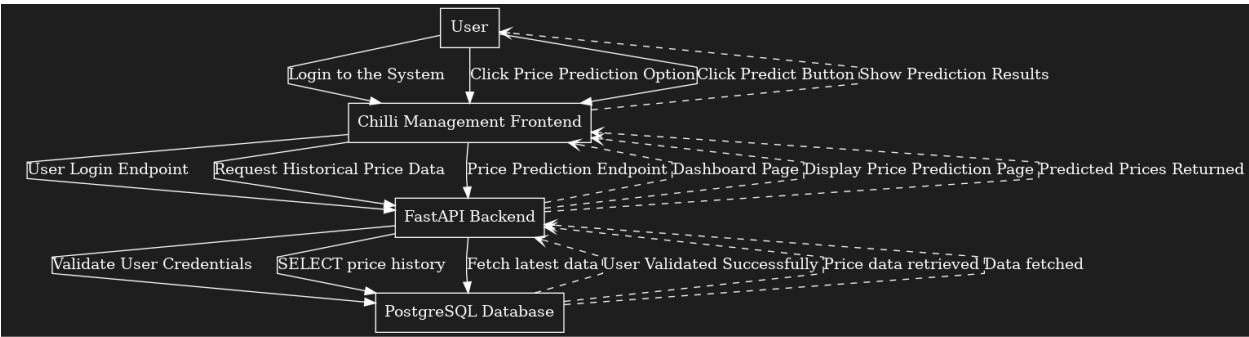


Figure 3 – Sequence Diagram

Design Narrative

The design narrative explains how and why the system’s architecture and components were chosen:

Multi-tier architecture was selected for separation of concerns and scalability.

FastAPI was chosen for its high performance and ease of integration with ML models.

React + Tailwind CSS provide a responsive, user-friendly interface.

PostgreSQL ensures reliable, structured storage for transaction, inventory, and user data.

TensorFlow + scikit-learn enable advanced price prediction, improving decision-making.

3.2 System Overview

The Chilli Management System is a web-based platform designed to streamline the chilli supply chain through secure role-based dashboards, real-time price prediction using machine learning, inventory tracking, and direct communication between suppliers and buyers. The system follows a multi-tier architecture, ensuring modularity, scalability, and maintainability.

Use Case Modeling

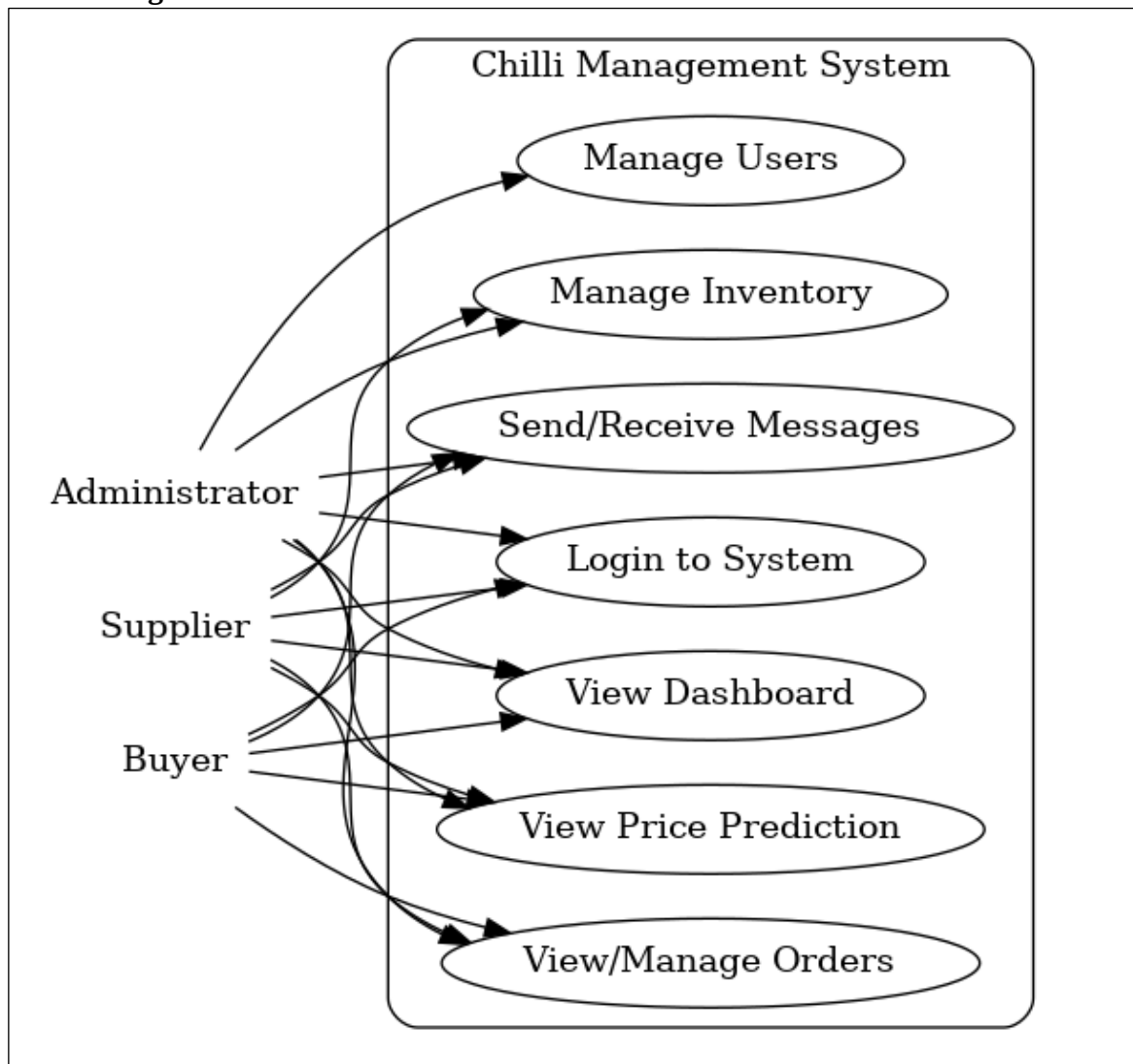


Figure 4 – Use Case Diagram

System Architecture

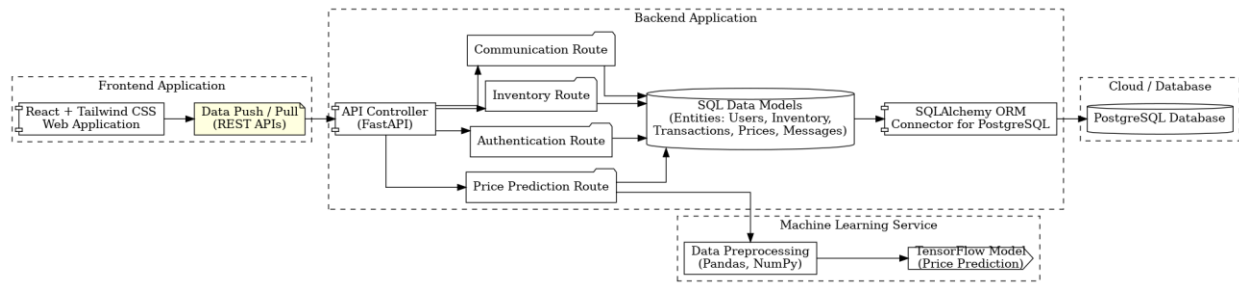


Figure 5 - System Architecture

Data Structure

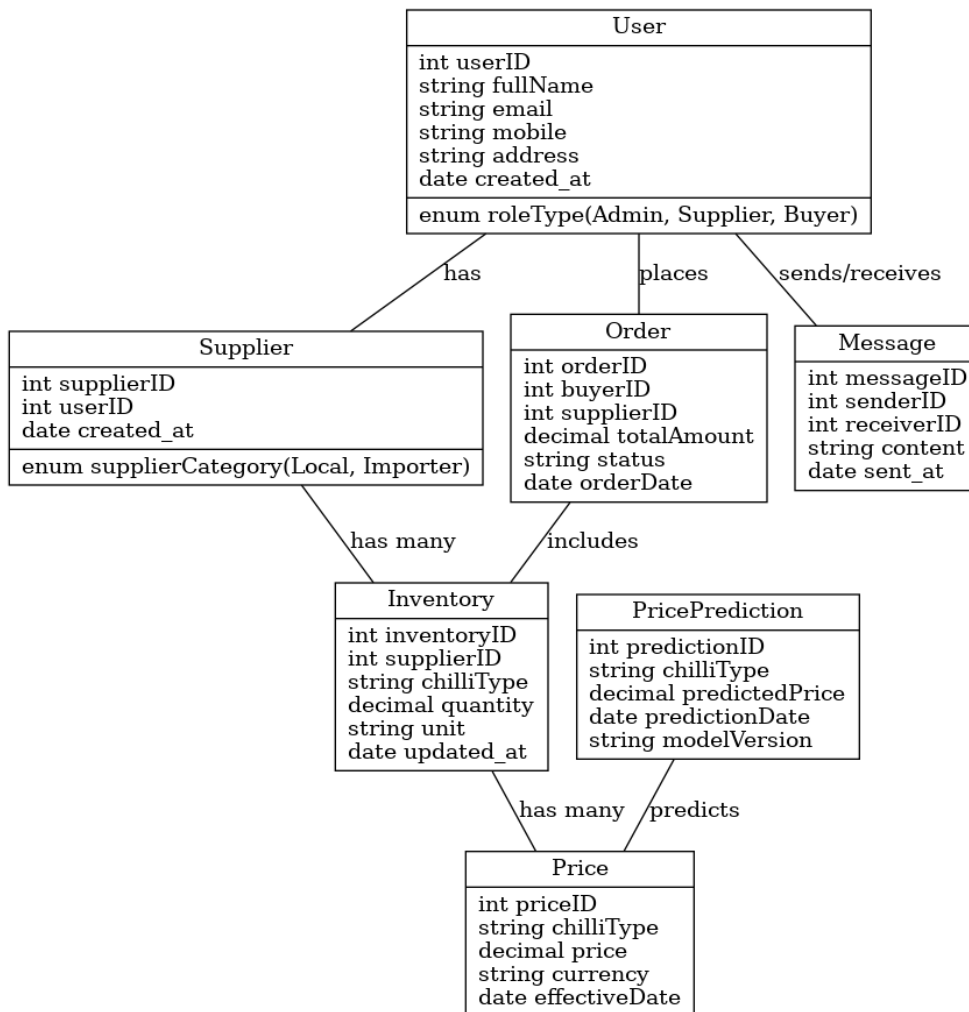


Figure 6 – Data Structure

Wireframes

Dashboard

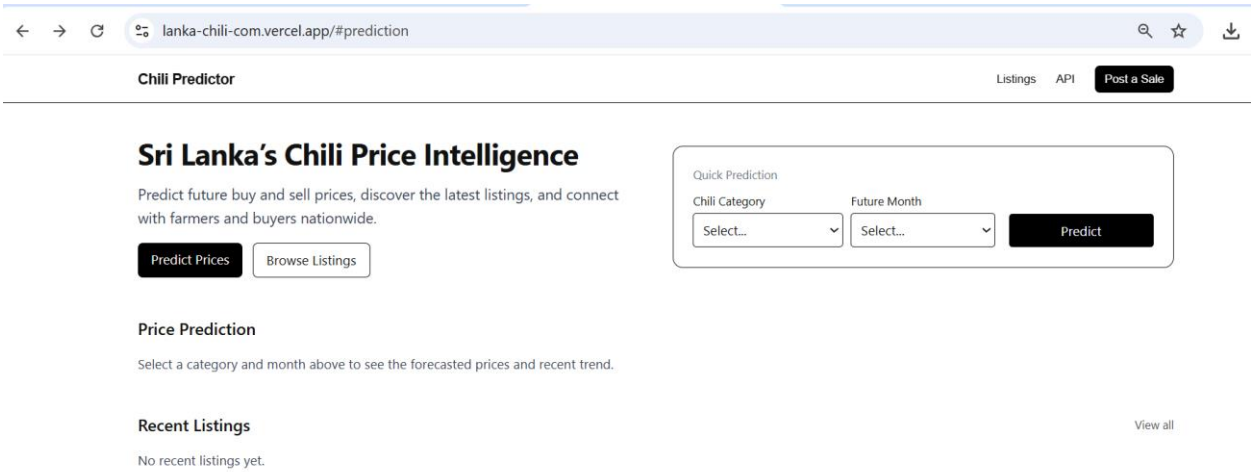


Figure 7 - System Dashboard

Chilli Price Predictor

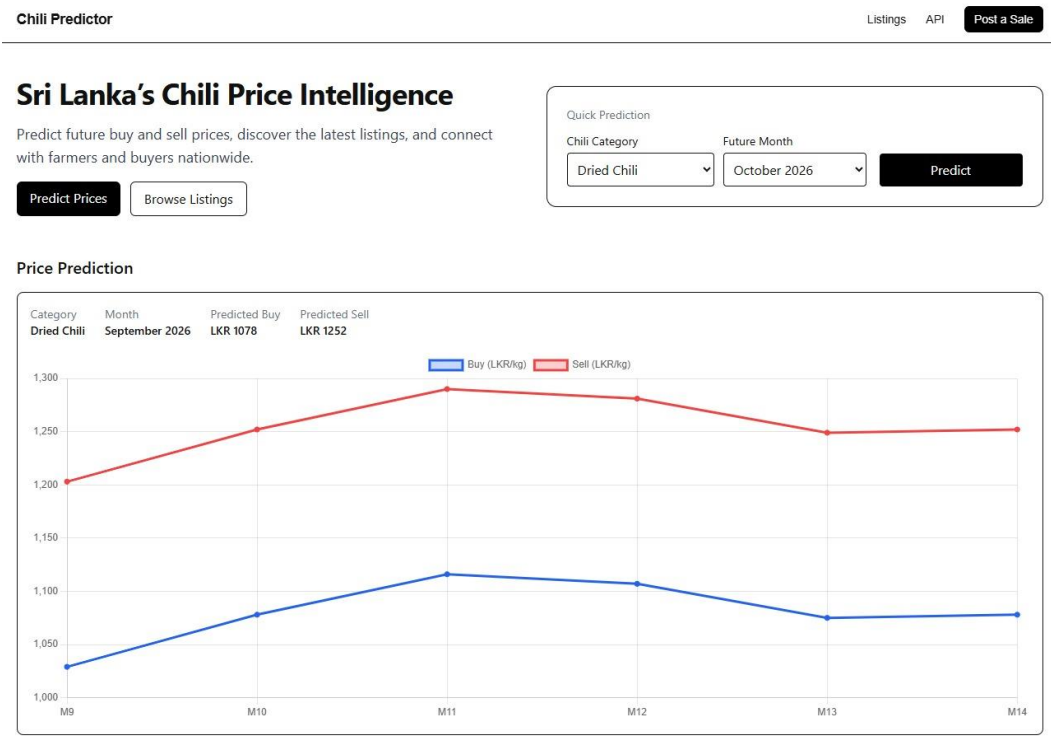


Figure 8 – Price Predictor

Chili Predictor API Docs

OpenAPI JSON: [/api/openapi](#)Chili Predictor API 1.0.0 OAS 3.0[/api/openapi](#)

API for predictions and chili sales

Servers

/

Authorize

predictions

POST /api/predict Predict chili buy/sell prices

sales

GET /api/sales List sales

POST /api/sales Create a sale

GET /api/sales/{id} Get sale by id

PATCH /api/sales/{id} Update sale

DELETE /api/sales/{id} Delete sale

Figure 9 - Predictions

Post a Sale

Sell your Chili

Chili Category

Select...

Amount (kg)

Price per kg (LKR)

District

Select...

Nearest town

Telephone

Notes (optional)

Post Sale

Figure 10 – Post a Sale

4. Product Implementation

4.1 Overview

The LankaChili Platform is a web-based price intelligence and marketplace prototype enabling chili suppliers and buyers in Sri Lanka to (a) forecast indicative future buy/sell prices, and (b) publish and browse current chili sale listings. The implementation prioritizes rapid MVP delivery, deterministic, predictable behavior for demonstrations, and an architecture that can evolve toward production (real ML model, role-based auth, analytics).

4.2 System Architecture

Layered structure:

- Presentation: Next.js (App Router) React components (Prediction form & chart, Listings, Sell form, API Docs page).
- API Layer: Next.js route handlers (/api/*) exposing prediction, reference data, and CRUD endpoints.
- Data & Integration: Supabase (PostgreSQL + potential Auth) or in-memory fallback store; external (optional) AI prediction endpoint abstraction.
- Client Libraries: Zod for schema validation; Chart.js for visualization; Supabase JS SDK for DB access.
- Security Boundary: API key (x-api-key) required for mutating sales operations; read endpoints public for transparency.

4.3 Technology Stack

- Front-End: Next.js 15, React 19, Tailwind CSS 4 (utility-first styling), Chart.js 4 for time-series visualization.
- Back-End: Node.js (embedded in Next.js runtime), Supabase (managed Postgres) or in-memory store for development.
- Validation: Zod schemas enforce structured, sanitized inputs.
- Documentation: Auto-served OpenAPI 3.0 spec + Swagger UI at /docs.
- Deployment Target (planned): Vercel or similar serverless Node platform; Supabase cloud service.

4.4 Module Descriptions

- Prediction Module (predictor.js): Provides deterministic mock price forecasts (seeded PRNG) and optional passthrough to external AI API if environment variables present. Returns final point + short historical trend sequence for chart.
- Sales Module (`/api/sales`, `/api/sales/[id]`): CRUD endpoints for chili listings; supports query filters (category, district). Handles dual persistence strategy (Supabase or in-memory). Includes server-side validation + timestamp management.
- Reference Data (`/api/categories`, `/api/locations`): Static enumerations for dropdowns and validation.
- OpenAPI Module (`/api/openapi`): Emits machine-readable spec consumed by Swagger UI page.
- Memory Store (memoryStore.js): Lightweight persistence substitute enabling offline/demo operation; isolates data layer to simplify later replacement.
- Supabase Client (supabase.js): Factory to produce service (server) and anon (future client) clients, centralizing configuration.

4.5 Data Model (Current)

- Entity: Sale Listing : id (int, PK), category (text enum), amountKg (numeric), pricePerKg (numeric), location (JSON: district, town), phone (varchar), notes (text optional), created_at (timestamp), updated_at (timestamp).
- Prediction Response (DTO) : category, month (ISO date), prediction { buy, sell }, history [{ monthOffset, buy, sell }].
- Reference Lists: categories[string], districts[string].
- (Planned Additions: user (auth), quality grade, harvest date, audit trails.)

4.6 API Endpoints (Implemented)

- POST /api/predict – returns forecast + history.
- GET /api/sales – list with optional ?category & ?district.
- POST /api/sales – create (auth header).
- GET /api/sales/{id} – detail.
- PATCH /api/sales/{id} – update (auth header).
- DELETE /api/sales/{id} – delete (auth header).
- GET /api/categories – category list.
- GET /api/locations – district list.
- GET /api/openapi – OpenAPI JSON.

4.7 User Interface Components

- Home Dashboard: Hero + quick prediction form + recent listings grid.
- Prediction Section: Displays numeric buy/sell forecast and Chart.js line chart of recent trend points.
- Sales Browse Page: Filter panel (category, district) + responsive cards presenting price, quantity, contact.
- Sell Form Page: Controlled form with validation, submission status messaging.
- API Docs Page: Embedded Swagger UI explorer for third-party integration readiness.

4.8 Security & Privacy Implementation (MVP)

- Write operations guarded by shared API key header.
- Validation rejects malformed or adversarial payloads before DB interaction.
- No storage of passwords or sensitive PII beyond phone number for direct communication.
- Environment variables (Supabase keys, external AI keys) kept outside code repository.

(Next Steps: per-user auth via Supabase Auth + RLS, rate limiting, audit logging.)

4.9 Performance & Optimization Practices

- Server-Side: Lightweight synchronous validation; minimal blocking I/O; fallback to in-memory reduces latency in demos.
- Client-Side: Lazy data fetch on mount; limited recent listings (≤ 8) on landing page; reused static reference data.
- Predictability: Deterministic mock prevents cache churn and facilitates reproducible UI states for testing.
- Bundling: Tailwind JIT reduces CSS payload; modular imports for Chart.js elements.

4.10 Development Workflow & Tooling

- ESLint ensures code consistency.
- Environment separation via presence/absence of Supabase keys.
- Readable modular files enabling unit test stubs (tests to be added in next sprint).

4.11 Progress vs Plan

- Completed: Core prediction mock, sales CRUD, filtering, reference data, UI pages, OpenAPI docs, basic security header.
- In Progress: Supabase production provisioning, improved error observability, initial unit tests.
- Pending / Future: Real ML model integration, role-based access, localization, analytics dashboards, automated test suite, performance monitoring, data export, multi-crop extension.

4.12 Current Limitations

- No persistent authentication or ownership of listings (any authorized key can mutate).
- Mock prediction not yet using real historical time-series.
- In-memory mode loses data on restart.
- Limited validation on phone number format and no rate limiting.

4.13 Risks & Mitigations

- Data Quality Risk: Relying on user-entered price/quantity → mitigate with future moderation & anomaly detection.
- Adoption Risk: Provide immediate value via prediction + simple posting flow; gather early stakeholder feedback.
- Security Risk: Shared API key → move to per-user tokens + RLS.
- Model Accuracy Risk: Interim mock replaced after sufficient historical dataset collection.

4.14 Deployment & Environment Strategy

- Target hosting: Vercel (auto build & deploy) + Supabase (managed Postgres/Auth).
- Environment Variables: NEXT_PUBLIC_SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, NEXT_PUBLIC_SUPABASE_ANON_KEY, AI_PREDICT_API_URL, AI_PREDICT_API_KEY.
- Promotion Flow: Local (in-memory) → Staging (Supabase test project) → Production (with RLS & backups).

5. Validation

5.1 Validation Objectives

- Confirm implemented features meet functional requirements (FR-1 to FR-23).
- Establish baseline performance, reliability, and usability metrics before expanding scope.
- De-risk transition from mock predictor to real ML service by ensuring stable API contract.

5.2 Validation Scope (Current Iteration)

- Functional: Prediction requests, listing creation, filtering, updating, deletion, reference data retrieval, OpenAPI availability.
- Non-Functional: Basic latency sampling, input validation behavior, deterministic prediction reproducibility.
- Excluded (Future): True model accuracy KPIs, large-scale load testing, security penetration tests, localization verification.

5.3 Methods & Techniques

- Static Analysis: ESLint pass (syntax / style conformance).
- Schema Validation: Zod-based runtime checks (negative tests crafted to trigger validation errors).
- API Contract Verification: Manual and Swagger UI “Try it out” calls verifying expected status codes & JSON shapes.
- Determinism Check: Repeated identical prediction requests produce identical outputs (seed test).
- CRUD Cycle Test: Create → List → Filter → Update → Get by id → Delete → Ensure 404 after deletion.
- Usability Heuristic Review: Nielsen heuristics quick pass (visibility of status, error prevention, minimal design).

- Performance Spot Tests: Record response times (cold vs warm) for /api/predict and /api/sales under light concurrency (browser + curl).
- Security Sanity: Attempt unauthorized POST/PATCH/DELETE without x-api-key (expect 401).
- Input Fuzzing (Light): Submit malformed JSON / missing fields to ensure structured 400 error format.

5.4 Test Data & Environment

- Environment: Local development (in-memory store) and staging Supabase (when configured).
- Sample Listings: Varied categories (Nai Miris, Kochchi, Green Chili) with realistic quantity (10–500 kg) and price ranges (LKR 700–1,500).
- Prediction Months: Future months spanning next 6–8 months to test date validation boundaries.

5.5 Key Test Cases (Illustrative)

- TC-01 Predict Valid: category="Nai Miris", month=FutureDate → 200 + prediction + non-empty history.
- TC-02 Predict Invalid Month: category valid, month=past date → 400 validation error.
- TC-03 Create Listing Unauthorized: Missing x-api-key → 401.
- TC-04 Create Listing Valid: All fields → 201 + persisted id.
- TC-05 Filter Listings: After multiple creates, /api/sales?district=Kandy returns only matching subset.
- TC-06 Update Listing: PATCH pricePerKg → 200 + updated field changed, updated_at refreshed.
- TC-07 Delete Listing: DELETE then GET returns 404.
- TC-08 OpenAPI Availability: GET /api/openapi returns JSON with paths for predict & sales.
- TC-09 Determinism: Two identical prediction POSTs return same buy/sell pair.

- TC-10 Validation Error Structure: Missing amountKg returns error object with field-level messages.

5.6 Results Summary (Current Snapshot)

- Functional Pass: All core CRUD and prediction endpoints operating as designed in local environment.
- Negative Tests: Invalid schema inputs reliably produce 400 with flattened error payload.
- Unauthorized Access: Blocked with 401 for write endpoints.
- Determinism: Confirmed identical outputs across 5 consecutive identical prediction requests.
- Latency (Indicative): Local /api/predict $\approx$ <100 ms mock; /api/sales list <120 ms (in-memory).
- Issues Identified: Lack of pagination (potential scalability issue), absence of rate limiting, no automated regression suite.

5.7 Performance & Load Validation (Initial)

- Approach: Manual concurrent browser tabs (≤ 10) issuing GET /api/sales; no observable degradation with in-memory store.
- Limitation: Not representative of production; formal k6 or Locust load test planned post persistence baseline.

5.8 Usability Findings

- Positive: Minimal fields, immediate feedback messages (“Created successfully”, “Predicting...”).
- Improvements: Add inline numeric range hints, mask/format for phone input, clearer empty state messaging for filters (“No listings match your criteria”).

5.9 Risk-Based Validation Gaps

- Model Accuracy: Not yet measured (mock). Plan: collect historical price + volume data; evaluate MAE / MAPE vs baseline naive seasonal model.
- Security Hardening: Pending penetration test & automated dependency scanning.
- Data Integrity: Need server-side district/category foreign key constraints once persistent DB finalized.

5.10 Defect Tracking & Resolution

- Current defects tracked informally (developer notes). Plan to introduce lightweight issue tracker (GitHub Issues) before next sprint to formalize severity, reproduction steps, and resolution timelines.

5.11 Acceptance Criteria Alignment

All MVP “Must” functional requirements validated through manual test cases (TC-01–TC-10). Non-functional acceptance partial: performance acceptable for demo; reliability & security depth tests scheduled.

5.12 Continuous Validation Plan (Next Phase)

- Introduce automated Jest test suite (unit tests for predictor deterministic output; integration tests hitting API routes with mock fetch).
- Add smoke test script to CI (build → run minimal API checks).
- Implement structured logging & metrics (response time histograms) for real usage telemetry.
- Conduct small stakeholder pilot (≤ 10 users) and gather structured feedback survey (task success, perceived accuracy).

5.13 Exit Criteria for Moving Beyond MVP

Automated test coverage $\geq 50\%$ of critical endpoints, zero open high-severity defects, successful Supabase persistence verification, approved usability adjustments, baseline model (or continued mock) delivering consistent performance metrics.

5.15 Coding Sample

Error State

```

4     'Kochchi',
5     'Green Chili',
6     'Dried Chili',
7     'Maldives Chili',
8   ]
9
10  export async function GET() {
11    return Response.json(
12      You, 22 hours ago * first commit ...
13  )

```

PROBLEMS **2** OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

node + - [] [X] [Y]

Caused by:

Syntax Error

Import trace for requested module:

- ./src/app/api/categories/route.js
- GET / 500 in 12187ms
- GET /api/sales 500 in 13975ms
- GET /api/categories 500 in 14044ms
- GET / 500 in 46ms
- GET / 500 in 20ms
- GET /_next/static/webpack/b088e1653816740e.webpack.hot-update.json 500 in 16ms

⚠ Fast Refresh had to perform a full reload. Read more: <https://nextjs.org/docs/messages/fast-refresh-reload>

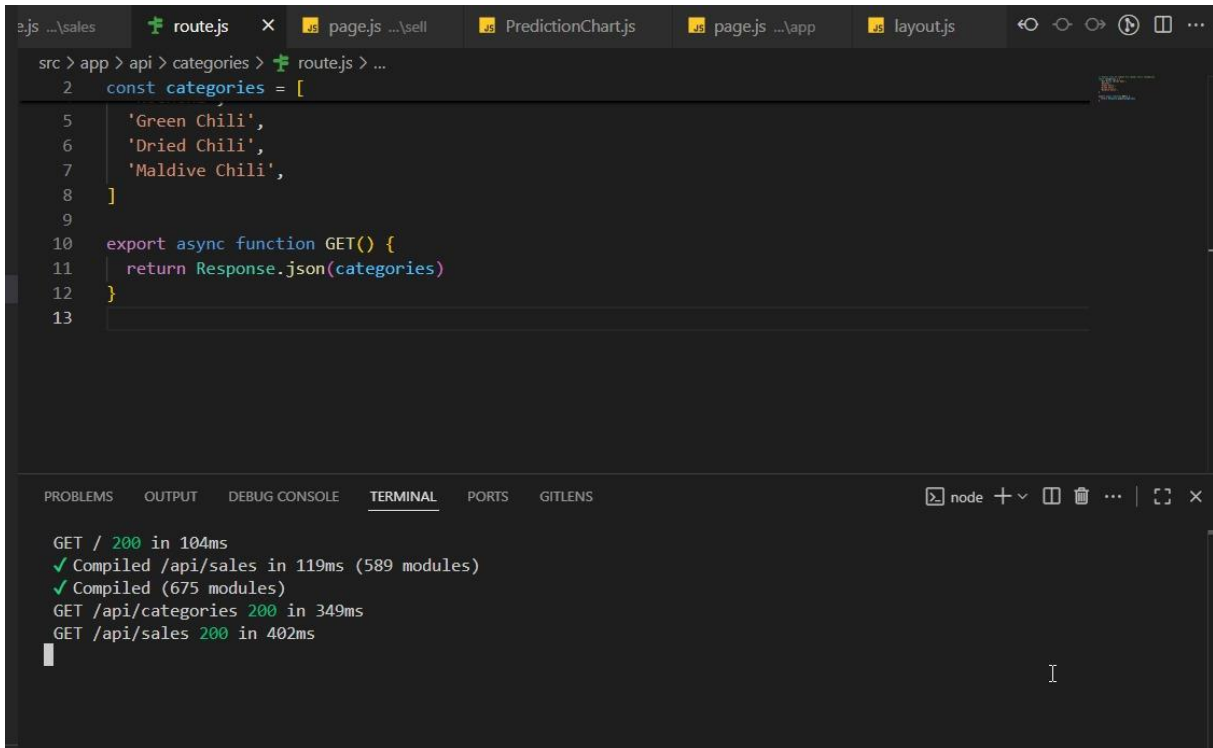
- GET / 200 in 16ms
- ⚠ Fast Refresh had to perform a full reload. Read more: <https://nextjs.org/docs/messages/fast-refresh-reload>
- GET / 200 in 16ms
- GET / 200 in 16ms
- GET / 500 in 18ms

✖ ./src/app/api/categories/route.js

Error: ✖ Expression expected

Figure 11 – Error State

Working State



The screenshot shows a VS Code editor with a dark theme. The top panel displays a file explorer with the following files: `route.js`, `page.js`, `PredictionChart.js`, `page.js`, and `layout.js`. The `route.js` file is open in the editor, showing the following code:

```
src > app > api > categories > route.js > ...  
2  const categories = [  
5    'Green Chili',  
6    'Dried Chili',  
7    'Maldiva Chili',  
8  ]  
9  
10 export async function GET() {  
11   return Response.json(categories)  
12 }  
13
```

The bottom panel shows the `TERMINAL` view with the following output:

```
node +  
GET / 200 in 104ms  
✓ Compiled /api/sales in 119ms (589 modules)  
✓ Compiled (675 modules)  
GET /api/categories 200 in 349ms  
GET /api/sales 200 in 402ms
```

Figure 12– Working Sample

5.15 Conclusion

Validation to date confirms architectural soundness and functional readiness of core marketplace + prediction flow. Remaining validation focuses on production-hardening (auth, persistence robustness, load, real model accuracy) prior to broader rollout.

6. Critical Review & Conclusion

6.1 Closing executive summary

This interim review confirms that the Chilli Management System project is progressing well toward its objectives of improving supply chain efficiency, enabling price prediction, and enhancing transparency. However, several refinements are recommended to strengthen the outcome. Adopting shorter Agile-style sprints alongside the current project management approach would improve responsiveness to stakeholder feedback. Expanding the dataset to include related crops could increase scalability, while parallelizing data collection and frontend development would reduce timeline risks. Formalizing a risk log and defining clear KPIs, such as prediction accuracy, system uptime, and user satisfaction, will provide measurable success criteria. These adjustments will enhance adaptability, mitigate delays, and ensure the system delivers both technical excellence and stakeholder value.

6.2 Conclusion

The Chilli Management System aims to improve supply chain efficiency, transparency, and price stability through role-based dashboards, inventory management, direct supplier–buyer communication, and machine learning–based price prediction. Progress indicates that these objectives remain achievable within the proposed timeline, supported by a robust architecture and modern technologies. Strengths include a clear problem scope, scalable design, and early attention to ethical and security considerations, while limitations involve data dependency and varying user digital literacy. Future work will focus on expanding to other commodities, adding a mobile application, integrating additional data sources, and conducting usability testing to refine the user experience.

References / Bibliography

Basnayake, B.R.P.M., Perera, R.S., and Jayasinghe, J.A.U.D., 2022. An approach for prediction of weekly prices of green chilli in Sri Lanka: Application of artificial neural network techniques. *Journal of Agricultural Sciences – Sri Lanka*, 17(3), pp.317–330.

Box, G.E.P. and Jenkins, G.M., 1970. *Time Series Analysis: Forecasting and Control*. San Francisco: Holden-Day.

Food and Agriculture Organization of the United Nations (FAO), 2023. *The State of Food and Agriculture 2023*. Rome: FAO.

World Food Programme (WFP), 2024. *Sri Lanka Market Monitor – Monthly Price Bulletin*. Available at: <https://www.wfp.org/publications/sri-lanka-market-monitor>.

MDPI, 2023. Predicting prices of staple crops using machine learning. *Agronomy*, 13(5), p.1125.

Nature Scientific Reports, 2025. Enhancing agricultural commodity price forecasting with deep learning. *Scientific Reports*, 15(1), p.22234.

Kaggle, 2024. Sri Lanka crop prices dataset. Available at: <https://www.kaggle.com/datasets>.

OpenAPI Initiative, 2024. *OpenAPI Specification 3.0*. Available at: <https://swagger.io/specification/>